

Formalization Seminar: Logic and Sets

George McNinch

Tufts University

2026-09-22

Welcome back	1
Plan for today	2
Bool vs Prop	3
Negation	6
Automating negation	10
Punchline, live	12
Wrap	14

Plan for today

Welcome back

- a quick word on `Bool` vs `Prop`

Plan for today

Welcome back

- a quick word on Bool vs Prop
- negation: what $\neg P$ actually **is**

Plan for today

Welcome back

- a quick word on Bool vs Prop
- negation: what $\neg P$ actually **is**
- automating negation: push_neg

- a quick word on Bool vs Prop
- negation: what $\neg P$ actually **is**
- automating negation: push_neg
- punchline, live: De Morgan for sets

Welcome back	1
Bool vs Prop	3
Why do we need this at all?	4
Two different kinds of things	5
Negation	6
Automating negation	10
Punchline, live	12
Wrap	14

Why do we need this at all?

Bool vs Prop

Why do we need this at all?

Bool vs Prop

- last time: Lean **checked** a proof, and it felt like a compilation step

Why do we need this at all?

Bool vs Prop

- last time: Lean **checked** a proof, and it felt like a compilation step
- but what, exactly, is it checking?

Why do we need this at all?

- last time: Lean **checked** a proof, and it felt like a compilation step
- but what, exactly, is it checking?
- some statements a computer really can just settle by calculation – $2 + 2 = 5$ is a finite check

Why do we need this at all?

- last time: Lean **checked** a proof, and it felt like a compilation step
- but what, exactly, is it checking?
- some statements a computer really can just settle by calculation – $2 + 2 = 5$ is a finite check
- most of what we actually care about can't be settled that way: “there are infinitely many primes,” “this sequence converges,” “these two groups are isomorphic” – there's no brute-force computation that decides these

Why do we need this at all?

Bool vs Prop

- last time: Lean **checked** a proof, and it felt like a compilation step
- but what, exactly, is it checking?
- some statements a computer really can just settle by calculation – $2 + 2 = 5$ is a finite check
- most of what we actually care about can't be settled that way: “there are infinitely many primes,” “this sequence converges,” “these two groups are isomorphic” – there's no brute-force computation that decides these
- Lean's answer: represent a statement as a **type** (Prop), and “this is true” means **producing a term of that type** – a genuine certificate, not a computed bit

Why do we need this at all?

- last time: Lean **checked** a proof, and it felt like a compilation step
- but what, exactly, is it checking?
- some statements a computer really can just settle by calculation – $2 + 2 = 5$ is a finite check
- most of what we actually care about can't be settled that way: “there are infinitely many primes,” “this sequence converges,” “these two groups are isomorphic” – there's no brute-force computation that decides these
- Lean's answer: represent a statement as a **type** (Prop), and “this is true” means **producing a term of that type** – a genuine certificate, not a computed bit
- this is the same idea behind last time's demo: `append_length` wasn't checked by running anything – it was checked by verifying the **proof term** has the right type

Two different kinds of things

Bool vs Prop

Two different kinds of things

Bool vs Prop

```
#eval (2 + 2 == 5)          -- Bool: computes, gives `false`  
#check (2 + 2 = 5 : Prop)   -- Prop: a statement
```

Two different kinds of things

Bool vs Prop

```
#eval (2 + 2 == 5)          -- Bool: computes, gives `false`  
#check (2 + 2 = 5 : Prop)   -- Prop: a statement
```

- Bool is ordinary **data** – two constructors, true and false. You **compute** a Bool.

Two different kinds of things

Bool vs Prop

```
#eval (2 + 2 == 5)          -- Bool: computes, gives `false`  
#check (2 + 2 = 5 : Prop)   -- Prop: a statement
```

- Bool is ordinary **data** – two constructors, true and false. You **compute** a Bool.
- Prop is the type of **statements**. To settle one you give a **proof**, not a value out of a two-element type.

Two different kinds of things

Bool vs Prop

```
#eval (2 + 2 == 5)          -- Bool: computes, gives `false`  
#check (2 + 2 = 5 : Prop)   -- Prop: a statement
```

- Bool is ordinary **data** – two constructors, true and false. You **compute** a Bool.
- Prop is the type of **statements**. To settle one you give a **proof**, not a value out of a two-element type.

```
example : ¬ (2 + 2 = 5) := by decide
```

Two different kinds of things

Bool vs Prop

```
#eval (2 + 2 == 5)          -- Bool: computes, gives `false`  
#check (2 + 2 = 5 : Prop)   -- Prop: a statement
```

- Bool is ordinary **data** – two constructors, true and false. You **compute** a Bool.
- Prop is the type of **statements**. To settle one you give a **proof**, not a value out of a two-element type.

```
example : ¬ (2 + 2 = 5) := by decide
```

- decide can settle statements like this automatically, because they're the kind of statement a computer can just check by calculation. (More on how, later in the semester.)

Welcome back	1
Bool vs Prop	3
Negation	6
What is $\neg P$, really?	7
Why is False absurd?	8
A note on names	9
Automating negation	10
Punchline, live	12
Wrap	14

What is $\neg P$, really?

What is $\neg P$, really?

- $\neg P$ is **notation** for $P \rightarrow \text{False}$

What is $\neg P$, really?

- $\neg P$ is **notation** for $P \rightarrow \text{False}$
- so “prove $\neg P$ ” means: construct a function taking a hypothetical proof of P and producing a proof of False

What is $\neg P$, really?

- $\neg P$ is **notation** for $P \rightarrow \text{False}$
- so “prove $\neg P$ ” means: construct a function taking a hypothetical proof of P and producing a proof of False

example : $\neg (1 = 0) := \text{by}$

```
intro h      -- h : 1 = 0, goal is now False
exact absurd h (by norm_num)
```

What is $\neg P$, really?

- $\neg P$ is **notation** for $P \rightarrow \text{False}$
- so “prove $\neg P$ ” means: construct a function taking a hypothetical proof of P and producing a proof of False

example : $\neg (1 = 0) := \text{by}$

```
intro h      -- h : 1 = 0, goal is now False
exact absurd h (by norm_num)
```

- `intro h` is the moment to linger on: it's the **same** `intro` you'd use on any implication – nothing special about negation as a connective, it's just an implication whose target happens to be False

What is $\neg P$, really?

- $\neg P$ is **notation** for $P \rightarrow \text{False}$
- so “prove $\neg P$ ” means: construct a function taking a hypothetical proof of P and producing a proof of False

example : $\neg (1 = 0) := \text{by}$

```
intro h      -- h : 1 = 0, goal is now False
exact absurd h (by norm_num)
```

- `intro h` is the moment to linger on: it’s the **same** `intro` you’d use on any implication – nothing special about negation as a connective, it’s just an implication whose target happens to be False
- this is exactly proof-by-contradiction, stated precisely: “assume P , derive an absurdity” **is** “give a term of type $P \rightarrow \text{False}$ ”

Why is False absurd?

- False is the type with **no constructors**

Why is False absurd?

- False is the type with **no constructors**
- so having a term of type False is absurd by construction: from one, you can derive anything (`h.elim`, `absurd`, `contradiction`)

Why is False absurd?

- False is the type with **no constructors**
- so having a term of type False is absurd by construction: from one, you can derive anything (`h.elim`, `absurd`, `contradiction`)
- that's what closes the loop – it's **why** $P \rightarrow \text{False}$ genuinely captures “P cannot hold”

A note on names

- Prop has its own True and False – these are **types**, not the Bool values from earlier

A note on names

- Prop has its own True and False – these are **types**, not the Bool values from earlier
- False : no constructors (as before – absurd)

A note on names

- Prop has its own True and False – these are **types**, not the Bool values from earlier
- False : no constructors (as before – absurd)
- True : one trivial constructor, `True.intro` – proving it needs no information at all

A note on names

- Prop has its own True and False – these are **types**, not the Bool values from earlier
- False : no constructors (as before – absurd)
- True : one trivial constructor, `True.intro` – proving it needs no information at all
- the capitalization is the tell: True/False (Prop) vs true/false (Bool) – you'll see both in Mathlib source, so it's worth having the distinction ready

Welcome back	1
Bool vs Prop	3
Negation	6
Automating negation	10
push_neg: De Morgan for propositions	11
Punchline, live	12
Wrap	14

- pushing a negation through by hand (intro, cases, ...) works, but gets tedious once quantifiers and connectives stack up

push_neg: De Morgan for propositions

- pushing a negation through by hand (intro, cases, ...) works, but gets tedious once quantifiers and connectives stack up

example (P Q : Prop) : $\neg (P \vee Q) \leftrightarrow \neg P \wedge \neg Q$:= **by**

push_neg

-- goal is closed, or reduced to something trivial

push_neg: De Morgan for propositions

- pushing a negation through by hand (intro, cases, ...) works, but gets tedious once quantifiers and connectives stack up

```
example (P Q : Prop) : ¬ (P ∨ Q) ↔ ¬P ∧ ¬Q := by
```

```
  push_neg
```

```
  -- goal is closed, or reduced to something trivial
```

- push_neg mechanically pushes negation through $\forall$, $\exists$, $\wedge$, $\vee$ – this **is** De Morgan, just at the level of propositions

push_neg: De Morgan for propositions

- pushing a negation through by hand (intro, cases, ...) works, but gets tedious once quantifiers and connectives stack up

example (P Q : Prop) : $\neg (P \vee Q) \leftrightarrow \neg P \wedge \neg Q$:= **by**

push_neg

-- goal is closed, or reduced to something trivial

- push_neg mechanically pushes negation through $\forall$, $\exists$, $\wedge$, $\vee$ – this **is** De Morgan, just at the level of propositions
- (we’re working classically throughout, so $\neg\neg P \rightarrow P$ is fair game – push_neg, by_contra, etc. rely on this)

Welcome back	1
Bool vs Prop	3
Negation	6
Automating negation	10
Punchline, live	12
De Morgan for sets	13
Wrap	14

- same shape of statement, now for sets:

```
example (s t : Set  $\alpha$ ) : (s  $\cup$  t)c = sc  $\cap$  tc := by
  ext x
  simp [Set.mem_union, Set.mem_compl_iff]
  push_neg
  tauto
```

- same shape of statement, now for sets:

```
example (s t : Set α) : (s ∪ t)c = sc ∩ tc := by
  ext x
  simp [Set.mem_union, Set.mem_compl_iff]
  push_neg
  tauto
```

- switching to the editor now – let's build this up piece by piece

- same shape of statement, now for sets:

```
example (s t : Set α) : (s ∪ t)c = sc ∩ tc := by
  ext x
  simp [Set.mem_union, Set.mem_compl_iff]
  push_neg
  tauto
```

- switching to the editor now – let’s build this up piece by piece
- watch for: `ext x` turns a **set equality** into an **iff of memberships** – this is where “sets” quietly becomes “logic” again

Welcome back	1
Bool vs Prop	3
Negation	6
Automating negation	10
Punchline, live	12
Wrap	14
Questions?	15

Questions?

Wrap

- where did the connection between logic and sets feel surprising?